

Chapter 2: Linear Algebra for Machine Learning

Master in Computer Science (M1) — Applications with Real Data in Python

Dr. Youssef SALMAN

Academic Year 2025–2026

Contents

1	Introduction	3
2	Vectors, Matrices and Tensors	3
2.1	Vectors	3
2.2	Matrices	5
2.3	Tensors	5
3	Matrix Arithmetic	7
3.1	Addition and Scalar Multiplication	7
3.2	Matrix Multiplication	8
3.3	Determinant	9
3.3.1	The space of square matrices	9
3.3.2	Definition of the determinant	10
3.3.3	Determinant of a 2×2 matrix	10
3.3.4	Geometric interpretation in $\mathbb{R}^2$	11
3.3.5	Geometric interpretation in $\mathbb{R}^3$	11
3.3.6	Volume-scaling interpretation	12
3.3.7	Determinant of a 3×3 matrix	12
3.3.8	Minors and cofactors	13
3.3.9	Main properties	13
3.3.10	Effect of elementary row operations	14
3.3.11	Triangular matrices	14
3.3.12	Determinant and linear independence	15
3.3.13	Interpretation in machine learning	15
3.4	Transpose	16
3.5	Inverse of a Matrix	17
3.5.1	Inverse of a 2×2 matrix	18
3.5.2	General formula using the adjugate matrix	18
3.5.3	Example: inverse of a 3×3 matrix	20
3.5.4	A singular matrix	22
3.5.5	Properties of the inverse	22

3.5.6	Solving a linear system	23
3.5.7	Application in machine learning	23
3.5.8	Numerical computation	24
3.6	Orthogonal matrix	25
4	Linear Transformations	25
4.1	Matrix Representation	26
4.2	Geometric Interpretations in $\mathbb{R}^2$	26
4.3	Kernel and Image	27
4.4	Rank and Invertibility	27
4.5	Composition of Transformations	27
4.6	Applications in Machine Learning	28
5	Eigenvalues and Eigenvectors	28
5.1	Existence	28
5.2	Computation	29
5.3	Multiplicity and Diagonalizability	29
5.4	Properties	29
5.5	Applications	30
6	Matrix Decompositions	30
6.1	Eigen-decomposition	30
6.2	Singular Value Decomposition (SVD)	32
6.2.1	Why do we need the SVD?	32
6.2.2	Definition of the SVD	32
6.2.3	Dimensions of the matrices	33
6.2.4	Meaning of U , Σ and V	34
6.2.5	Geometric interpretation	34
6.2.6	Relationship with eigenvalues	35
6.2.7	Steps for calculating the SVD	36
6.2.8	Short calculation example	37
6.2.9	SVD as a sum of rank-one matrices	38
6.2.10	Rank and singular values	39
6.2.11	Conditioning and numerical stability	39
6.2.12	Moore–Penrose pseudoinverse	39
6.2.13	Low-rank approximation	40
6.2.14	Choosing the number of components	41
6.2.15	Importance of SVD in machine learning	42
6.2.16	SVD in Python	44
7	Application: Principal Component Analysis (PCA)	49
7.1	Geometric and Statistical Intuition	49
7.2	Algorithmic Steps	50
7.3	Worked Numerical Example (2D Data)	50
7.4	Interpretation in Machine Learning	53
8	Exercise - Blood Pressure Data: PCA Interpretation	53
9	Conclusion	54

1 Introduction

Machine learning is the field of computer science that focuses on building algorithms capable of learning patterns from data and making predictions or decisions without being explicitly programmed. From recommending movies on streaming platforms to detecting anomalies in network traffic or recognizing faces in images, machine learning is now embedded in daily technology.

Linear algebra, on the other hand, is a branch of mathematics concerned with vectors, matrices, and linear transformations. It provides the formal language to describe high-dimensional data and the operations we perform on them. Whenever we work with a dataset of thousands of features, compute a weighted sum of inputs, or rotate data into a new coordinate system, we are using the principles of linear algebra.

These two worlds come together naturally: machine learning relies heavily on linear algebra as its computational backbone. Training a regression model involves solving systems of linear equations; deep learning requires millions of matrix multiplications; dimensionality reduction methods such as Principal Component Analysis (PCA) depend on eigenvalues and eigenvectors. Without linear algebra, machine learning would lack the tools to represent, manipulate, and transform data efficiently.

Real applications illustrate this connection clearly. Search engines use matrix decompositions (such as the singular value decomposition) to index billions of documents. Image compression standards rely on linear transformations to reduce file size without losing essential details. Recommendation systems use vector similarity to suggest products or movies that are close to a user's past choices. In each of these cases, linear algebra not only underlies the algorithm but also determines its performance and scalability.

This chapter is therefore devoted to exploring the linear algebra concepts most relevant for machine learning. We begin with the basic objects, vectors and matrices, and study their arithmetic. We then interpret matrices as linear transformations and examine their structure through eigenvalues and eigenvectors. Finally, we introduce matrix decompositions, including the singular value decomposition, and show how these ideas culminate in one of the most widely used techniques in data science: Principal Component Analysis (PCA).

2 Vectors, Matrices and Tensors

2.1 Vectors

Definition 2.1. A **vector** is an ordered collection of numbers representing a single object with several characteristics (features).

A vector with d components is written as

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \in \mathbb{R}^d.$$

Each component represents one feature of the object.

Example (Machine Learning).

Suppose a house is described by area (120 m²), number of bedrooms (3), age (15 years), distance from the city center (8 km). The house can be represented by the vector

$$\mathbf{x} = \begin{bmatrix} 120 \\ 3 \\ 15 \\ 8 \end{bmatrix}.$$

Each row corresponds to one feature used by a machine learning algorithm.

The most common vector operations are:

- **Euclidean norm**

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_d^2},$$

which measures the length of the vector.

- **Dot product**

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^d x_i y_i,$$

which measures how strongly two vectors are aligned.

- **Cosine similarity**

$$\cos(\theta) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|},$$

widely used in information retrieval, recommendation systems and natural language processing.

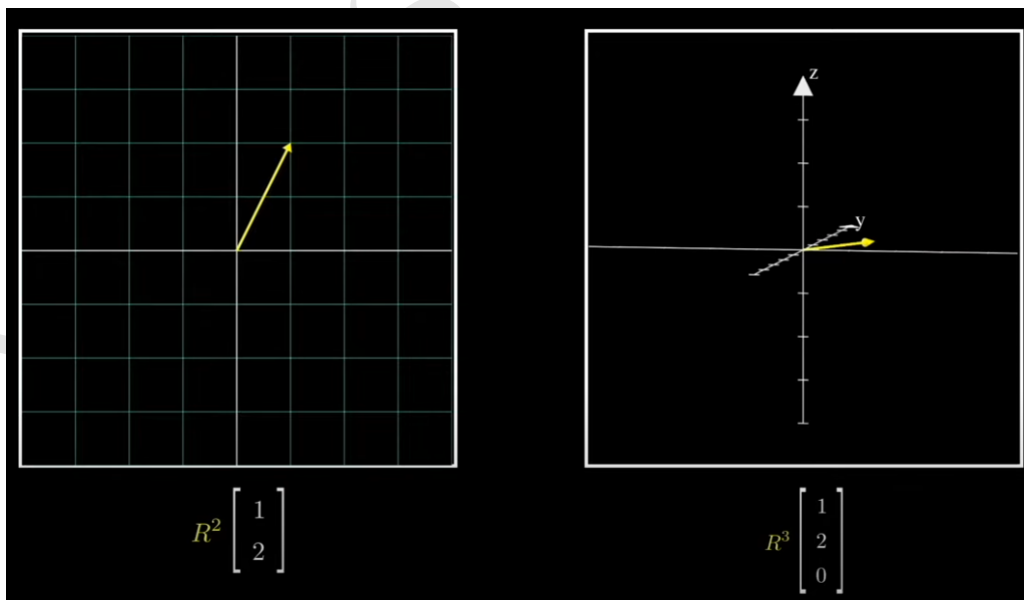


Figure 1: Vectors in a two-dimensional space.

2.2 Matrices

Definition 2.2. A **matrix** is a rectangular array of numbers organized into rows and columns.

An $m \times n$ matrix is written as

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

Matrices represent collections of vectors or linear transformations.

Example (Dataset).

Suppose information is collected from four students.

$$X = \begin{bmatrix} 170 & 65 & 22 \\ 180 & 78 & 24 \\ 165 & 60 & 20 \\ 175 & 70 & 23 \end{bmatrix}$$

where

- each row represents one student,
- each column represents one feature:
 - height (cm),
 - weight (kg),
 - age (years).

In machine learning, datasets are almost always stored as matrices.

If

$$A \in \mathbb{R}^{m \times n}, \quad \mathbf{x} \in \mathbb{R}^n,$$

then

$$A\mathbf{x} \in \mathbb{R}^m,$$

is called the **matrix-vector product**. It computes weighted combinations of the components of $\mathbf{x}$ and is the basic operation used in neural networks.

2.3 Tensors

Definition 2.3. A **tensor** is a multidimensional array that generalizes vectors and matrices.

- A scalar is a tensor of order 0.
- A vector is a tensor of order 1.

- A matrix is a tensor of order 2.
- Higher-order tensors have three or more dimensions.

For example,

$$T \in \mathbb{R}^{3 \times 4 \times 2}$$

is a third-order tensor containing

$$3 \times 4 \times 2 = 24$$

values.

Example (Color Image).

A color image of size

$$256 \times 256$$

contains three color channels:

- Red,
- Green,
- Blue (RGB).

Therefore, it is represented as the tensor

$$256 \times 256 \times 3.$$

Each pixel contains three numerical values corresponding to the intensity of the red, green and blue colors.

Example (Video).

A video is a sequence of images. If a video contains 100 frames of size

$$256 \times 256 \times 3,$$

then it can be represented as the tensor

$$100 \times 256 \times 256 \times 3.$$

The four dimensions correspond to

- frame number,
- image height,
- image width,
- color channel.

Deep learning libraries such as **TensorFlow** and **PyTorch** manipulate data almost exclusively as tensors.

3 Matrix Arithmetic

3.1 Addition and Scalar Multiplication

Let

$$A = (a_{ij}), \quad B = (b_{ij}) \in \mathbb{R}^{m \times n}.$$

The matrices A and B can be added only when they have the same number of rows and columns.

Their sum is defined componentwise by

$$A + B = (a_{ij} + b_{ij}).$$

Equivalently,

$$(A + B)_{ij} = a_{ij} + b_{ij}.$$

For a scalar $\lambda \in \mathbb{R}$, scalar multiplication is defined by

$$\lambda A = (\lambda a_{ij}).$$

Thus,

$$(\lambda A)_{ij} = \lambda a_{ij}.$$

Example. Let

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 1 \\ -2 & 3 \end{bmatrix}.$$

Then

$$A + B = \begin{bmatrix} 1+5 & 2+1 \\ 3-2 & 4+3 \end{bmatrix} = \begin{bmatrix} 6 & 3 \\ 1 & 7 \end{bmatrix}.$$

Moreover,

$$2A = \begin{bmatrix} 2 & 4 \\ 6 & 8 \end{bmatrix}.$$

Interpretation in machine learning. Matrix addition can be used to combine datasets, parameter updates or intermediate outputs of a model.

Scalar multiplication is used, for example, when adjusting the learning rate in gradient-based optimization:

$$W_{\text{new}} = W_{\text{old}} - \eta \nabla L(W_{\text{old}}),$$

where η is the learning rate.

3.2 Matrix Multiplication

Matrix multiplication

Let

$$A \in \mathbb{R}^{m \times n}, \quad B \in \mathbb{R}^{n \times p}.$$

The product

$$C = AB$$

is the matrix in $\mathbb{R}^{m \times p}$ whose entries are

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}.$$

Thus, the entry c_{ij} is obtained by taking the dot product of the i -th row of A with the j -th column of B .

The multiplication is possible because the number of columns of A is equal to the number of rows of B :

$$\underbrace{A}_{m \times n} \underbrace{B}_{n \times p} = \underbrace{AB}_{m \times p}.$$

Example. Let

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 0 \\ 1 & 5 \end{bmatrix}.$$

Then

$$AB = \begin{bmatrix} 1(2) + 2(1) & 1(0) + 2(5) \\ 3(2) + 4(1) & 3(0) + 4(5) \end{bmatrix} = \begin{bmatrix} 4 & 10 \\ 10 & 20 \end{bmatrix}.$$

Matrix multiplication is associative:

$$(AB)C = A(BC),$$

but it is generally not commutative:

$$AB \neq BA.$$

Interpretation as a composition of transformations. Suppose that B transforms a vector $\mathbf{x}$ and that A then transforms the result. We have

$$\mathbf{x} \mapsto B\mathbf{x} \mapsto A(B\mathbf{x}).$$

Since

$$A(B\mathbf{x}) = (AB)\mathbf{x},$$

the matrix AB represents the composition of the two transformations.

Application in machine learning. A fully connected neural-network layer is written as

$$\mathbf{z} = W\mathbf{x} + \mathbf{b},$$

where

- $\mathbf{x}$ is the input vector;
- W is the matrix of weights;
- $\mathbf{b}$ is the bias vector;
- $\mathbf{z}$ is the output vector.

Thus, matrix multiplication is one of the fundamental operations in machine learning and deep learning.

3.3 Determinant

3.3.1 The space of square matrices

Let $\mathbb{K}$ denote either $\mathbb{R}$ or $\mathbb{C}$.

The notation

$$M_n(\mathbb{K})$$

denotes the set of all square matrices of size $n \times n$ with entries in $\mathbb{K}$:

$$M_n(\mathbb{K}) = \{A = (a_{ij}) : a_{ij} \in \mathbb{K}, 1 \leq i, j \leq n\}.$$

The determinant is defined only for square matrices.

Therefore, if

$$A \in \mathbb{R}^{m \times n} \quad \text{with} \quad m \neq n,$$

then $\det(A)$ is not defined.

3.3.2 Definition of the determinant

Determinant of a square matrix

Let $n \geq 1$. The determinant is the unique function

$$\det : M_n(\mathbb{K}) \longrightarrow \mathbb{K}$$

satisfying the following properties:

1. it is linear with respect to each column;
2. exchanging two columns changes the sign of the determinant;
- 3.

$$\det(I_n) = 1,$$

where I_n is the identity matrix.

For $A \in M_n(\mathbb{K})$, the scalar $\det(A)$ is called the **determinant of A** .

A determinant can be written in either of the following forms:

$$\det(A) \quad \text{or} \quad \begin{vmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{vmatrix}.$$

The determinant assigns a single number to a square matrix. This number summarizes several important properties of the corresponding linear transformation.

3.3.3 Determinant of a 2×2 matrix

For

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix},$$

the determinant is

$$\det(A) = \begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc.$$

Example. Let

$$A = \begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix}.$$

Then

$$\det(A) = 3(4) - 2(1) = 12 - 2 = 10.$$

3.3.4 Geometric interpretation in $\mathbb{R}^2$

Consider two vectors

$$\mathbf{u} = \begin{bmatrix} a \\ c \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} b \\ d \end{bmatrix}.$$

Place these vectors as the columns of the matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}.$$

Then

$$|\det(A)|$$

is the area of the parallelogram generated by $\mathbf{u}$ and $\mathbf{v}$.
Hence,

$$\text{Area} = |\det(A)|.$$

The sign of the determinant describes orientation:

- $\det(A) > 0$: orientation is preserved;
- $\det(A) < 0$: orientation is reversed;
- $\det(A) = 0$: the parallelogram collapses into a line.

If $\det(A) = 0$, the two column vectors are linearly dependent.

3.3.5 Geometric interpretation in $\mathbb{R}^3$

Let

$$A = \begin{bmatrix} | & | & | \\ \mathbf{u} & \mathbf{v} & \mathbf{w} \\ | & | & | \end{bmatrix} \in M_3(\mathbb{R}).$$

Then

$$|\det(A)|$$

is the volume of the parallelepiped generated by $\mathbf{u}$, $\mathbf{v}$ and $\mathbf{w}$.
Equivalently,

$$\det(A) = (\mathbf{u} \times \mathbf{v}) \cdot \mathbf{w}.$$

• If

$$\det(A) = 0,$$

then the three vectors lie in the same plane and are linearly dependent.

3.3.6 Volume-scaling interpretation

More generally, if a square matrix A represents a linear transformation, then

$$|\det(A)|$$

is the factor by which the transformation changes areas or volumes.

For example:

- if $|\det(A)| = 2$, volumes are multiplied by 2;
- if $|\det(A)| = \frac{1}{2}$, volumes are divided by 2;
- if $\det(A) = 0$, the transformation collapses the space into a lower-dimensional subspace.

Therefore, the determinant measures how much a linear transformation expands, contracts or collapses space.

3.3.7 Determinant of a 3×3 matrix

Let

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}.$$

Expanding along the first row gives

$$\det(A) = a \begin{vmatrix} e & f \\ h & i \end{vmatrix} - b \begin{vmatrix} d & f \\ g & i \end{vmatrix} + c \begin{vmatrix} d & e \\ g & h \end{vmatrix}.$$

Consequently,

$$\det(A) = a(ei - fh) - b(di - fg) + c(dh - eg).$$

Example. Let

$$A = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 1 & 4 \\ 2 & 0 & 1 \end{bmatrix}.$$

Expanding along the first row,

$$\begin{aligned} \det(A) &= 1 \begin{vmatrix} 1 & 4 \\ 0 & 1 \end{vmatrix} - 2 \begin{vmatrix} 3 & 4 \\ 2 & 1 \end{vmatrix} + 0 \begin{vmatrix} 3 & 1 \\ 2 & 0 \end{vmatrix} \\ &= 1(1) - 2(3 - 8) \\ &= 1 + 10 \\ &= 11. \end{aligned}$$

3.3.8 Minors and cofactors

Minor and cofactor
Let $A = (a_{ij}) \in M_n(\mathbb{K}).$ The minor M_{ij} is the determinant of the matrix obtained by deleting the i-th row and the j-th column of A. The corresponding cofactor is $C_{ij} = (-1)^{i+j} M_{ij}.$

The determinant can be expanded along any row i :

$$\det(A) = \sum_{j=1}^n a_{ij} C_{ij}.$$

It can also be expanded along any column j :

$$\det(A) = \sum_{i=1}^n a_{ij} C_{ij}.$$

In practice, one should choose a row or column containing many zeros.

3.3.9 Main properties

For $A, B \in M_n(\mathbb{K})$ and $\lambda \in \mathbb{K}$, the determinant satisfies:

$$\det(I_n) = 1,$$

$$\det(AB) = \det(A) \det(B),$$

$$\det(A^T) = \det(A),$$

and

$$\det(\lambda A) = \lambda^n \det(A).$$

If A is invertible, then

$$\det(A^{-1}) = \frac{1}{\det(A)}.$$

A square matrix is invertible if and only if

$$\det(A) \neq 0.$$

• Thus,

$$A \text{ is invertible} \iff \det(A) \neq 0.$$

Similarly,

$$A \text{ is singular} \iff \det(A) = 0.$$

3.3.10 Effect of elementary row operations

The determinant changes according to the row operation performed.

- 1. Exchanging two rows** If two rows are exchanged,

$$R_i \leftrightarrow R_j,$$

the determinant changes sign.

$$\det(B) = -\det(A).$$

- 2. Multiplying one row by a scalar** If one row is multiplied by λ ,

$$R_i \leftarrow \lambda R_i,$$

then the determinant is also multiplied by λ :

$$\det(B) = \lambda \det(A).$$

- 3. Adding a multiple of one row to another** If

$$R_i \leftarrow R_i + \lambda R_j, \quad i \neq j,$$

then the determinant does not change:

$$\det(B) = \det(A).$$

The same rules hold for column operations.

3.3.11 Triangular matrices

If A is upper triangular or lower triangular, its determinant is the product of its diagonal entries:

$$\det(A) = \prod_{i=1}^n a_{ii}.$$

For example,

$$A = \begin{bmatrix} 2 & 1 & 4 \\ 0 & 3 & 5 \\ 0 & 0 & -2 \end{bmatrix}$$

is upper triangular, so

$$\det(A) = 2(3)(-2) = -12.$$

This property makes Gaussian elimination an efficient method for computing determinants.

3.3.12 Determinant and linear independence

Let

$$A = \begin{bmatrix} | & & | \\ \mathbf{a}_1 & \cdots & \mathbf{a}_n \\ | & & | \end{bmatrix}.$$

Then

$$\det(A) \neq 0$$

if and only if the column vectors

$$\mathbf{a}_1, \dots, \mathbf{a}_n$$

are linearly independent.

Equivalently,

$$\det(A) = 0$$

if and only if at least one column is a linear combination of the others.

Therefore,

$$\det(A) \neq 0 \iff \text{rank}(A) = n.$$

3.3.13 Interpretation in machine learning

The determinant is useful in several areas of machine learning.

1. Detecting redundant features If the columns of a design matrix contain exact linear dependencies, then

$$\det(X^T X) = 0.$$

In this case, $X^T X$ is singular and cannot be inverted.

This situation occurs when one feature can be written as an exact linear combination of other features.

2. Multicollinearity When

$$|\det(X^T X)|$$

is very close to zero, the columns of X are nearly linearly dependent.

This may indicate strong multicollinearity and can make estimated regression coefficients unstable.

3. Multivariate Gaussian distributions For a multivariate normal distribution with covariance matrix Σ , the density contains the term

$$\frac{1}{\sqrt{\det(\Sigma)}}.$$

The determinant $\det(\Sigma)$ measures the generalized variance or the overall spread of the data.

4. Change of variables When data are transformed by

$$\mathbf{y} = A\mathbf{x},$$

the corresponding volume element is multiplied by

$$|\det(A)|.$$

This appears in probability density transformations and generative models.

3.4 Transpose

Transpose of a matrix

Let

$$A = (a_{ij}) \in \mathbb{R}^{m \times n}.$$

The transpose of A , denoted by A^T , is the matrix in $\mathbb{R}^{n \times m}$ defined by

$$(A^T)_{ij} = a_{ji}.$$

It is obtained by interchanging the rows and columns of A .

For example, if

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix},$$

then

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

The main properties of the transpose are

$$(A^T)^T = A,$$

$$(A + B)^T = A^T + B^T,$$

$$(\lambda A)^T = \lambda A^T,$$

and

$$(AB)^T = B^T A^T.$$

Notice that the order is reversed in the transpose of a product.

Symmetric matrices. A square matrix A is called symmetric if

$$A^T = A.$$

Covariance matrices and Gram matrices such as

$$X^T X$$

are symmetric.

Application in machine learning. Suppose

$$X \in \mathbb{R}^{m \times n},$$

where rows represent observations and columns represent features.

Then

$$X^T X \in \mathbb{R}^{n \times n}$$

measures relationships between features, while

$$X X^T \in \mathbb{R}^{m \times m}$$

measures relationships between observations.

3.5 Inverse of a Matrix

Inverse of a matrix

Let

$$A \in M_n(\mathbb{K}).$$

The inverse of A , denoted by A^{-1} , is a matrix satisfying

$$AA^{-1} = A^{-1}A = I_n.$$

If such a matrix exists, A is called **invertible** or **nonsingular**. Otherwise, A is called **singular**.

Not every square matrix has an inverse.

The following statements are equivalent:

A is invertible,

$$\det(A) \neq 0,$$

$$\text{rank}(A) = n,$$

and

the columns of A are linearly independent.

3.5.1 Inverse of a 2×2 matrix

Let

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}.$$

If

$$\det(A) = ad - bc \neq 0,$$

then

$$A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Example. Let

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}.$$

First,

$$\det(A) = 2(1) - 1(1) = 1.$$

Therefore,

$$A^{-1} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}.$$

Indeed,

$$AA^{-1} = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

3.5.2 General formula using the adjugate matrix

The inverse of a general square matrix can be calculated using its determinant, minors and cofactors.

Let

$$A = (a_{ij}) \in M_n(\mathbb{K}).$$

● For every entry a_{ij} , the corresponding **minor** M_{ij} is the determinant of the matrix obtained by deleting the i -th row and the j -th column of A .

The corresponding **cofactor** is

$$C_{ij} = (-1)^{i+j} M_{ij}.$$

The signs of the cofactors follow the alternating pattern

$$\begin{bmatrix} + & - & + & \cdots \\ - & + & - & \cdots \\ + & - & + & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}.$$

The matrix of cofactors is

$$C = \begin{bmatrix} C_{11} & C_{12} & \cdots & C_{1n} \\ C_{21} & C_{22} & \cdots & C_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ C_{n1} & C_{n2} & \cdots & C_{nn} \end{bmatrix}.$$

Adjugate matrix

The **adjugate matrix** of A , denoted by $\text{adj}(A)$, is the transpose of the cofactor matrix:

$$\text{adj}(A) = C^T.$$

Therefore,

$$\text{adj}(A) = \begin{bmatrix} C_{11} & C_{21} & \cdots & C_{n1} \\ C_{12} & C_{22} & \cdots & C_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ C_{1n} & C_{2n} & \cdots & C_{nn} \end{bmatrix}.$$

The matrix A satisfies

$$A \text{adj}(A) = \text{adj}(A)A = \det(A)I_n.$$

Consequently, when

$$\det(A) \neq 0,$$

the general formula for the inverse is

$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A)$$

or, equivalently,

$$A^{-1} = \frac{1}{\det(A)} C^T.$$

Thus, the general procedure for finding the inverse is:

1. calculate $\det(A)$;
2. verify that $\det(A) \neq 0$;
3. calculate all minors M_{ij} ;

4. calculate the cofactors

$$C_{ij} = (-1)^{i+j} M_{ij};$$

5. form the cofactor matrix C ;

6. transpose C to obtain $\text{adj}(A)$;

7. use

$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A).$$

3.5.3 Example: inverse of a 3×3 matrix

Consider the matrix

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix}.$$

We calculate its inverse using the adjugate formula.

Step 1: Calculate the determinant. Expanding along the first row,

$$\begin{aligned} \det(A) &= 1 \begin{vmatrix} 1 & 4 \\ 6 & 0 \end{vmatrix} - 2 \begin{vmatrix} 0 & 4 \\ 5 & 0 \end{vmatrix} + 3 \begin{vmatrix} 0 & 1 \\ 5 & 6 \end{vmatrix} \\ &= 1(1(0) - 4(6)) - 2(0(0) - 4(5)) + 3(0(6) - 1(5)) \\ &= -24 + 40 - 15 \\ &= 1. \end{aligned}$$

Since

$$\det(A) = 1 \neq 0,$$

the matrix is invertible.

Step 2: Calculate the minors. The minors of the first row are

$$M_{11} = \begin{vmatrix} 1 & 4 \\ 6 & 0 \end{vmatrix} = -24,$$

$$M_{12} = \begin{vmatrix} 0 & 4 \\ 5 & 0 \end{vmatrix} = -20,$$

and

$$M_{13} = \begin{vmatrix} 0 & 1 \\ 5 & 6 \end{vmatrix} = -5.$$

The minors of the second row are

$$M_{21} = \begin{vmatrix} 2 & 3 \\ 6 & 0 \end{vmatrix} = -18,$$

$$M_{22} = \begin{vmatrix} 1 & 3 \\ 5 & 0 \end{vmatrix} = -15,$$

and

$$M_{23} = \begin{vmatrix} 1 & 2 \\ 5 & 6 \end{vmatrix} = 6 - 10 = -4.$$

The minors of the third row are

$$M_{31} = \begin{vmatrix} 2 & 3 \\ 1 & 4 \end{vmatrix} = 8 - 3 = 5,$$

$$M_{32} = \begin{vmatrix} 1 & 3 \\ 0 & 4 \end{vmatrix} = 4,$$

and

$$M_{33} = \begin{vmatrix} 1 & 2 \\ 0 & 1 \end{vmatrix} = 1.$$

Therefore, the matrix of minors is

$$M = \begin{bmatrix} -24 & -20 & -5 \\ -18 & -15 & -4 \\ 5 & 4 & 1 \end{bmatrix}.$$

Step 3: Calculate the cofactors. Using

$$C_{ij} = (-1)^{i+j} M_{ij},$$

and the sign pattern

$$\begin{bmatrix} + & - & + \\ - & + & - \\ + & - & + \end{bmatrix},$$

we obtain

$$C = \begin{bmatrix} -24 & 20 & -5 \\ 18 & -15 & 4 \\ 5 & -4 & 1 \end{bmatrix}.$$

Step 4: Calculate the adjugate matrix. The adjugate is the transpose of the cofactor matrix:

$$\text{adj}(A) = C^T.$$

Therefore,

$$\text{adj}(A) = \begin{bmatrix} -24 & 18 & 5 \\ 20 & -15 & -4 \\ -5 & 4 & 1 \end{bmatrix}.$$

Step 5: Calculate the inverse. Using

$$A^{-1} = \frac{1}{\det(A)} \operatorname{adj}(A),$$

and since $\det(A) = 1$, we obtain

$$A^{-1} = \begin{bmatrix} -24 & 18 & 5 \\ 20 & -15 & -4 \\ -5 & 4 & 1 \end{bmatrix}.$$

Verification. Indeed,

$$AA^{-1} = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 4 \\ 5 & 6 & 0 \end{bmatrix} \begin{bmatrix} -24 & 18 & 5 \\ 20 & -15 & -4 \\ -5 & 4 & 1 \end{bmatrix}.$$

Calculating the product gives

$$AA^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = I_3.$$

3.5.4 A singular matrix

Consider

$$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}.$$

Its determinant is

$$\det(B) = 1(4) - 2(2) = 0.$$

Therefore, B is singular and does not have an inverse.

Its second row is twice its first row, so the rows are linearly dependent.

Geometrically, the transformation represented by B maps the plane onto a line. Information is lost, and the transformation cannot be reversed.

3.5.5 Properties of the inverse

For invertible matrices A and B ,

$$(AB)^{-1} = B^{-1}A^{-1},$$

$$(A^T)^{-1} = (A^{-1})^T,$$

$$(A^{-1})^{-1} = A,$$

and

$$(\lambda A)^{-1} = \frac{1}{\lambda} A^{-1}, \quad \lambda \neq 0.$$

Moreover,

$$\det(A^{-1}) = \frac{1}{\det(A)}.$$

The inverse of a product reverses the order of the matrices. For example,

$$(ABC)^{-1} = C^{-1}B^{-1}A^{-1}.$$

3.5.6 Solving a linear system

Consider the system

$$A\mathbf{x} = \mathbf{b}.$$

If A is invertible, multiplying both sides by A^{-1} gives

$$A^{-1}A\mathbf{x} = A^{-1}\mathbf{b}.$$

Since

$$A^{-1}A = I_n,$$

we obtain

$$I_n\mathbf{x} = A^{-1}\mathbf{b}.$$

Therefore,

$$\boxed{\mathbf{x} = A^{-1}\mathbf{b}}.$$

This solution is unique because an invertible matrix represents a one-to-one linear transformation.

However, in numerical computation, linear systems are generally solved without explicitly calculating the inverse.

3.5.7 Application in machine learning

The closed-form ordinary least-squares estimator is formally written as

$$\hat{\boldsymbol{\beta}} = (X^T X)^{-1} X^T \mathbf{y},$$

provided that $X^T X$ is invertible.

If the columns of X are linearly dependent, then

$$\det(X^T X) = 0,$$

and the inverse does not exist.

- This means that the regression coefficients cannot be uniquely determined using the ordinary inverse.

If the columns are nearly linearly dependent, $X^T X$ may be close to singular, leading to numerical instability and coefficients that are very sensitive to small changes in the data.

Ridge regression addresses this problem by replacing $X^T X$ with

$$X^T X + \lambda I, \quad \lambda > 0.$$

The corresponding estimator is

$$\hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T \mathbf{y}.$$

The addition of λI increases the diagonal entries and generally makes the matrix invertible and numerically more stable.

3.5.8 Numerical computation

Although the adjugate formula

$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A)$$

is important theoretically, it is not generally used to invert large matrices in practical machine-learning applications.

Calculating all minors and cofactors is computationally expensive when the dimension of the matrix is large.

In practice, one should also avoid explicitly computing

$$(X^T X)^{-1}.$$

More stable methods include:

- **QR decomposition:**

$$X = QR,$$

where Q has orthonormal columns and R is upper triangular. The least-squares problem becomes

$$R\hat{\beta} = Q^T \mathbf{y}.$$

Since R is triangular, the coefficients can be found efficiently by back substitution.

- **Singular Value Decomposition:**

$$X = U\Sigma V^T.$$

The SVD reveals the rank of X and allows the use of the Moore–Penrose pseudoinverse:

$$X^+ = V\Sigma^+ U^T.$$

The least-squares solution is then

$$\hat{\beta} = X^+ \mathbf{y}.$$

QR decomposition is efficient for full-rank, well-conditioned problems, while SVD is more robust when the matrix is ill-conditioned or rank deficient.

3.6 Orthogonal matrix

Orthogonal matrix

A square matrix

$$Q \in \mathbb{R}^{n \times n}$$

is called an **orthogonal matrix** if its transpose is equal to its inverse, that is,

$$Q^T Q = Q Q^T = I_n.$$

Equivalently,

$$Q^{-1} = Q^T.$$

An orthogonal matrix has columns (and rows) that are mutually orthogonal unit vectors. In other words,

$$\mathbf{q}_i^T \mathbf{q}_j = \begin{cases} 1, & \text{if } i = j, \\ 0, & \text{if } i \neq j, \end{cases}$$

where $\mathbf{q}_1, \dots, \mathbf{q}_n$ denote the columns of Q .

Orthogonal matrices preserve vector lengths and angles. For every vector $\mathbf{x} \in \mathbb{R}^n$,

$$\|Q\mathbf{x}\| = \|\mathbf{x}\|.$$

Therefore, an orthogonal matrix represents a rigid transformation, such as a rotation or a reflection, without changing the shape or size of an object.

Example. The matrix

$$Q = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

is orthogonal because

$$Q^T Q = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I_2.$$

This matrix represents a rotation of 90° counterclockwise in the plane.

4 Linear Transformations

Linear transformation

A **linear transformation** is a mapping $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ that preserves both vector addition and scalar multiplication:

$$T(\mathbf{x} + \mathbf{y}) = T(\mathbf{x}) + T(\mathbf{y}), \quad T(\alpha\mathbf{x}) = \alpha T(\mathbf{x}),$$

for all vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and scalars $\alpha \in \mathbb{R}$.

4.1 Matrix Representation

Every linear transformation can be represented as multiplication by a matrix. That is, for some $A \in \mathbb{R}^{m \times n}$ we can write

$$T(\mathbf{x}) = A\mathbf{x}.$$

Conversely, every matrix defines a linear transformation. This equivalence allows us to study transformations through the properties of their matrices.

4.2 Geometric Interpretations in $\mathbb{R}^2$

Multiplying by a matrix can produce different geometric effects:

- **Scaling:** $T(\mathbf{x}) = \alpha\mathbf{x}$ enlarges or shrinks vectors.
- **Reflection:** $T(x, y) = (x, -y)$ reflects points across the x -axis.
- **Rotation:**

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

rotates vectors counterclockwise by angle θ .

- **Horizontal shear:** A horizontal shear shifts each point horizontally by an amount proportional to its vertical coordinate.

$$S = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}.$$

For a point

$$\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix},$$

we have

$$S\mathbf{x} = \begin{bmatrix} x + ky \\ y \end{bmatrix}.$$

Thus,

$$(x, y) \mapsto (x + ky, y).$$

The value of y remains unchanged, while the point is shifted horizontally. Geometrically, a rectangle or square is transformed into a parallelogram.

- **Projection:**

$$P = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

projects vectors onto the x -axis, collapsing all y components to zero.

4.3 Kernel and Image

Given a linear transformation $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$, two fundamental subspaces describe its behavior.

Kernel

The **kernel** (or null space) of T is the set of all input vectors that are mapped to zero:

$$\ker(T) = \{\mathbf{x} \in \mathbb{R}^n \mid T(\mathbf{x}) = 0\}.$$

If the kernel is only the zero vector, then T is one-to-one (injective).

Image

The **image** (or range) of T is the set of all possible outputs:

$$\text{Im}(T) = \{T(\mathbf{x}) \mid \mathbf{x} \in \mathbb{R}^n\}.$$

If the image is the whole $\mathbb{R}^m$, then T is onto (surjective).

Together, the kernel and image describe how much information is lost or preserved by the transformation.

4.4 Rank and Invertibility

Rank and Invertibility

The **rank** of T is the dimension of its image. It measures how many independent directions are preserved by the transformation.

If $T : \mathbb{R}^n \rightarrow \mathbb{R}^n$ and $\text{rank}(T) = n$, then the transformation is both injective and surjective, hence invertible. Its inverse T^{-1} is also linear.

Nullity Theorem.

For every linear transformation $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$,

$$\dim(\ker T) + \dim(\text{Im } T) = n.$$

This fundamental identity links the null space and the image. For example, if the kernel is large, the image must be correspondingly smaller.

4.5 Composition of Transformations

The composition of two linear transformations corresponds to matrix multiplication. If $T_1(\mathbf{x}) = A\mathbf{x}$ and $T_2(\mathbf{x}) = B\mathbf{x}$, then

$$T_2(T_1(\mathbf{x})) = B(A\mathbf{x}) = (BA)\mathbf{x}.$$

This shows that matrix multiplication encodes the chaining of transformations in the correct order.

Example (Projection). Consider the projection

$$P = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

It satisfies $P^2 = P$, meaning repeated application does nothing new (idempotent). Moreover, $P^T = P$, so P is an **orthogonal projection** onto the x -axis.

4.6 Applications in Machine Learning

Linear transformations are everywhere in machine learning:

- In **neural networks**, each layer applies $T(\mathbf{x}) = W\mathbf{x} + b$, a linear transformation followed by a nonlinearity.
- In **dimensionality reduction**, PCA projects data onto a lower-dimensional subspace using a linear transformation defined by principal components.
- In **computer vision**, rotations, scalings, and reflections are modeled by linear transformations on pixel vectors.
- In **feature engineering**, normalization and standardization can be expressed as linear transformations.

5 Eigenvalues and Eigenvectors

Eigenvector

Let A be a square $n \times n$ matrix. A nonzero vector $\mathbf{v} \in \mathbb{R}^n$ (or $\mathbb{C}^n$) is called an **eigenvector** of A if there exists a scalar λ such that

$$A\mathbf{v} = \lambda\mathbf{v}.$$

The scalar λ is the corresponding **eigenvalue**.

Geometrically, eigenvectors point in directions that are preserved under the transformation A , while eigenvalues indicate the factor by which these directions are stretched or shrunk.

5.1 Existence

Not every real matrix has real eigenvalues, but every square matrix has eigenvalues if we allow complex numbers. This is a consequence of the Fundamental Theorem of Algebra: the characteristic polynomial of degree n has n (possibly complex) roots.

- Real symmetric (or Hermitian) matrices have n real eigenvalues.
- Orthogonal matrices have eigenvalues of modulus 1.
- Diagonal matrices have eigenvalues equal to their diagonal entries.

5.2 Computation

To find eigenvalues, we solve

$$(A - \lambda I)\mathbf{v} = 0 \iff \det(A - \lambda I) = 0.$$

This determinant equation is called the **characteristic equation**. Its roots are the eigenvalues, and the null space of $(A - \lambda I)$ gives the eigenvectors.

Example 1 (real eigenvalues).

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}.$$

The characteristic polynomial is $(2 - \lambda)(3 - \lambda)$, giving eigenvalues $\lambda_1 = 2, \lambda_2 = 3$ with eigenvectors along the x - and y -axes.

Example 2 (complex eigenvalues).

$$B = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}.$$

This is a 90° rotation. Its characteristic polynomial is $\lambda^2 + 1 = 0$, so eigenvalues are $\lambda = i, -i$. No real eigenvectors exist, but over $\mathbb{C}$ we can find eigenvectors.

5.3 Multiplicity and Diagonalizability

Each eigenvalue has an **algebraic multiplicity** (how many times it is a root of the characteristic polynomial) and a **geometric multiplicity** (dimension of its eigenspace).

Always: geometric multiplicity $\leq$ algebraic multiplicity.

Definition 5.1. A matrix is **diagonalizable** if there exists a basis of eigenvectors. Symmetric matrices are always diagonalizable.

5.4 Properties

- The sum of eigenvalues (with multiplicity) equals the trace: $\text{tr}(A) = \sum \lambda_i$.
- The product of eigenvalues equals the determinant: $\det(A) = \prod \lambda_i$.
- Eigenvectors associated with distinct eigenvalues of a symmetric matrix are orthogonal.
- If A is invertible, its eigenvalues are all nonzero, and the eigenvalues of A^{-1} are λ_i^{-1} .
- If A is triangular, its eigenvalues are simply the diagonal entries.

5.5 Applications

Eigenvalues and eigenvectors appear throughout machine learning and computer science:

- **Principal Component Analysis (PCA):** eigenvectors of the covariance matrix give directions of maximum variance.
- **Spectral clustering:** eigenvectors of graph Laplacians identify community structure in networks.
- **PageRank:** the importance of web pages is given by the dominant eigenvector of a stochastic matrix.
- **Stability analysis:** in dynamical systems, signs of eigenvalues of the system matrix determine stability.

6 Matrix Decompositions

Matrix decomposition consists of expressing a matrix as a product of simpler matrices.

The purpose is similar to factoring a number. For example,

$$12 = 3 \times 4.$$

In linear algebra, a complicated matrix can often be represented using matrices with useful structures, such as diagonal, triangular, or orthogonal matrices.

Matrix decompositions are important in machine learning because they can simplify:

- the solution of linear systems;
- least-squares estimation;
- dimensionality reduction;
- data compression;
- noise removal;
- the analysis of large datasets.

6.1 Eigen-decomposition

Suppose that

$$A \in \mathbb{R}^{n \times n}$$

has n linearly independent eigenvectors.

- Then A is diagonalizable and can be written as

$$A = PDP^{-1},$$

where

$$P = \begin{bmatrix} | & & | \\ \mathbf{v}_1 & \cdots & \mathbf{v}_n \\ | & & | \end{bmatrix}$$

contains the eigenvectors of A as columns, and

$$D = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}$$

contains the corresponding eigenvalues.

Thus,

$$A\mathbf{v}_i = \lambda_i\mathbf{v}_i.$$

The decomposition

$$A = PDP^{-1}$$

means that the transformation represented by A can be understood in three steps:

1. use P^{-1} to express a vector in the eigenvector basis;
2. use D to scale each eigenvector direction by its eigenvalue;
3. use P to return to the original coordinate system.

Eigen-decomposition is useful for computing powers of a matrix because

$$A^k = PD^kP^{-1},$$

where

$$D^k = \begin{bmatrix} \lambda_1^k & 0 & \cdots & 0 \\ 0 & \lambda_2^k & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n^k \end{bmatrix}.$$

However, eigen-decomposition has two important limitations:

- it is defined only for square matrices;
- not every square matrix is diagonalizable.

The Singular Value Decomposition avoids these limitations.

6.2 Singular Value Decomposition (SVD)

6.2.1 Why do we need the SVD?

In machine learning, datasets are commonly represented by rectangular matrices:

$$A \in \mathbb{R}^{m \times n},$$

where

- m is the number of observations;
- n is the number of features.

Such a matrix is not necessarily square, so an eigen-decomposition of A may not be possible.

The Singular Value Decomposition can be applied to **every** real matrix, whether it is:

- square or rectangular;
- invertible or singular;
- full rank or rank deficient.

6.2.2 Definition of the SVD

Singular Value Decomposition

Let

$$A \in \mathbb{R}^{m \times n}.$$

The Singular Value Decomposition of A is a factorization

$$A = U \Sigma V^T$$

where:

•

$$U \in \mathbb{R}^{m \times m}$$

is an orthogonal matrix;

•

$$V \in \mathbb{R}^{n \times n}$$

is an orthogonal matrix;

•

$$\Sigma \in \mathbb{R}^{m \times n}$$

is a rectangular diagonal matrix containing nonnegative numbers called the **singular values** of A .

Since U and V are orthogonal,

$$U^T U = U U^T = I_m$$

and

$$V^T V = V V^T = I_n.$$

Therefore,

$$U^{-1} = U^T \quad \text{and} \quad V^{-1} = V^T.$$

The matrix Σ has the form

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_r \\ 0 & 0 & \cdots & 0 \end{bmatrix},$$

where

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0.$$

The numbers

$$\sigma_1, \sigma_2, \dots, \sigma_r$$

are the nonzero singular values of A .

6.2.3 Dimensions of the matrices

For

$$A \in \mathbb{R}^{m \times n},$$

the dimensions in the full SVD are

$$\underbrace{A}_{m \times n} = \underbrace{U}_{m \times m} \underbrace{\Sigma}_{m \times n} \underbrace{V^T}_{n \times n}.$$

The dimensions are compatible because

$$(m \times m)(m \times n)(n \times n) = m \times n.$$

If the rank of A is r , the reduced or thin SVD is

$$A = U_r \Sigma_r V_r^T$$

- where

$$U_r \in \mathbb{R}^{m \times r}, \quad \Sigma_r \in \mathbb{R}^{r \times r}, \quad V_r \in \mathbb{R}^{n \times r}.$$

The reduced SVD stores only the components corresponding to nonzero singular values.

6.2.4 Meaning of U , Σ and V

Let

$$U = \begin{bmatrix} | & & | \\ \mathbf{u}_1 & \cdots & \mathbf{u}_m \\ | & & | \end{bmatrix}$$

and

$$V = \begin{bmatrix} | & & | \\ \mathbf{v}_1 & \cdots & \mathbf{v}_n \\ | & & | \end{bmatrix}.$$

Then:

- the vectors $\mathbf{u}_i$ are called the **left singular vectors**;
- the vectors $\mathbf{v}_i$ are called the **right singular vectors**;
- the numbers σ_i are called the **singular values**.

They satisfy

$$A\mathbf{v}_i = \sigma_i\mathbf{u}_i.$$

Thus, when the matrix A acts on the right singular vector $\mathbf{v}_i$, the result points in the direction $\mathbf{u}_i$ and has its length multiplied by σ_i .

Similarly,

$$A^T\mathbf{u}_i = \sigma_i\mathbf{v}_i.$$

The singular values measure the strength of the transformation in the corresponding singular-vector directions.

A large singular value represents an important direction, while a small singular value represents a direction containing relatively little information.

6.2.5 Geometric interpretation

The decomposition

$$A = U\Sigma V^T$$

describes a linear transformation as three successive operations.

For a vector $\mathbf{x}$,

$$A\mathbf{x} = U\Sigma V^T\mathbf{x}.$$

- The operations occur from right to left:

1.

$$V^T\mathbf{x}$$

rotates or reflects the input coordinate system;

2.

$$\Sigma V^T \mathbf{x}$$

stretches or contracts the coordinate directions using the singular values;

3.

$$U \Sigma V^T \mathbf{x}$$

rotates or reflects the result into the output space.

Therefore, every linear transformation can be interpreted as

$$\boxed{\text{rotation/reflection} \longrightarrow \text{scaling} \longrightarrow \text{rotation/reflection}}.$$

For example, in two dimensions, the unit circle is transformed by A into an ellipse.

The right singular vectors give the principal directions of the input space, the singular values give the lengths of the ellipse axes, and the left singular vectors give the directions of the ellipse axes in the output space.

6.2.6 Relationship with eigenvalues

The SVD can be calculated using the symmetric matrices

$$A^T A \quad \text{and} \quad A A^T.$$

Starting from

$$A = U \Sigma V^T,$$

we obtain

$$\begin{aligned} A^T A &= (U \Sigma V^T)^T (U \Sigma V^T) \\ &= V \Sigma^T U^T U \Sigma V^T \\ &= V \Sigma^T \Sigma V^T. \end{aligned}$$

Since $U^T U = I_m$,

$$\boxed{A^T A = V \Sigma^T \Sigma V^T}.$$

Therefore:

- the columns of V are eigenvectors of $A^T A$;
- the eigenvalues of $A^T A$ are

$$\sigma_1^2, \sigma_2^2, \dots;$$

- the singular values are the square roots of these eigenvalues:

$$\boxed{\sigma_i = \sqrt{\lambda_i(A^T A)}}.$$

Similarly,

$$\boxed{A A^T = U \Sigma \Sigma^T U^T}.$$

Therefore, the columns of U are eigenvectors of $A A^T$.

6.2.7 Steps for calculating the SVD

Let

$$A \in \mathbb{R}^{m \times n}.$$

To compute

$$A = U\Sigma V^T,$$

the following steps can be used.

Step 1: Calculate $A^T A$. Form the symmetric matrix

$$A^T A \in \mathbb{R}^{n \times n}.$$

Step 2: Find the eigenvalues of $A^T A$. Solve

$$\det(A^T A - \lambda I_n) = 0.$$

Let the eigenvalues be ordered as

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0.$$

The eigenvalues are nonnegative because

$$\mathbf{x}^T A^T A \mathbf{x} = \|A\mathbf{x}\|^2 \geq 0.$$

Step 3: Calculate the singular values. The singular values are

$$\sigma_i = \sqrt{\lambda_i}.$$

They are conventionally ordered as

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0.$$

Place them on the diagonal of Σ .

Step 4: Find the right singular vectors. For each eigenvalue λ_i , solve

$$(A^T A - \lambda_i I_n) \mathbf{v}_i = \mathbf{0}.$$

Normalize each eigenvector so that

$$\|\mathbf{v}_i\| = 1.$$

Then form

$$V = \begin{bmatrix} | & & | \\ \mathbf{v}_1 & \cdots & \mathbf{v}_n \\ | & & | \end{bmatrix}.$$

The eigenvectors must be placed in the same order as their corresponding singular values.

Step 5: Find the left singular vectors. For every nonzero singular value σ_i , calculate

$$\mathbf{u}_i = \frac{A\mathbf{v}_i}{\sigma_i}.$$

The vectors $\mathbf{u}_i$ are normalized and form the columns of U .

If additional columns are needed for the full SVD, complete the collection using orthonormal vectors.

Step 6: Assemble the decomposition. Finally, write

$$A = U\Sigma V^T.$$

The result can be verified by multiplying the three matrices.

6.2.8 Short calculation example

Consider

$$A = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}.$$

First,

$$A^T A = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 9 & 0 \\ 0 & 4 \end{bmatrix}.$$

The eigenvalues of $A^T A$ are

$$\lambda_1 = 9, \quad \lambda_2 = 4.$$

Therefore, the singular values are

$$\sigma_1 = \sqrt{9} = 3, \quad \sigma_2 = \sqrt{4} = 2.$$

The normalized eigenvectors of $A^T A$ are

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Thus,

$$V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

The left singular vectors are

$$\mathbf{u}_1 = \frac{A\mathbf{v}_1}{\sigma_1} = \frac{1}{3} \begin{bmatrix} 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

and

$$\mathbf{u}_2 = \frac{A\mathbf{v}_2}{\sigma_2} = \frac{1}{2} \begin{bmatrix} 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Hence,

$$U = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

and

$$\Sigma = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}.$$

Therefore,

$$A = U\Sigma V^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

This example is simple because A is already diagonal. For a general matrix, the singular vectors are usually not the standard basis vectors.

6.2.9 SVD as a sum of rank-one matrices

Let

$$A = U\Sigma V^T$$

and let

$$r = \text{rank}(A).$$

Then the SVD can also be written as

$$A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T.$$

More explicitly,

$$A = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^T + \cdots + \sigma_r \mathbf{u}_r \mathbf{v}_r^T.$$

Each matrix

$$\mathbf{u}_i \mathbf{v}_i^T$$

has rank one.

Thus, the SVD expresses A as a sum of rank-one components ordered by importance:

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r.$$

The first component is the strongest component, the second is the next strongest component, and so on.

6.2.10 Rank and singular values

The rank of a matrix is equal to the number of nonzero singular values:

$$\text{rank}(A) = \#\{i : \sigma_i > 0\}.$$

Therefore:

$$A \text{ has full column rank} \iff \sigma_n > 0$$

when $m \geq n$.

For a square matrix,

$$A \text{ is invertible} \iff \sigma_i > 0 \text{ for every } i.$$

If at least one singular value is zero, then the matrix is singular.

Very small singular values also indicate that the matrix is close to being singular.

6.2.11 Conditioning and numerical stability

For a full-rank matrix, the condition number based on the singular values is

$$\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}} = \frac{\sigma_1}{\sigma_r}.$$

Interpretation:

- if $\kappa(A)$ is close to 1, the matrix is well-conditioned;
- if $\kappa(A)$ is very large, the matrix is ill-conditioned;
- if $\sigma_r = 0$, then A is singular and

$$\kappa(A) = \infty.$$

An ill-conditioned matrix can amplify small errors in the data or small rounding errors in numerical calculations.

This is important in linear regression because strong multicollinearity produces very small singular values in the design matrix.

6.2.12 Moore–Penrose pseudoinverse

The SVD provides a natural generalization of the matrix inverse.

Suppose

$$A = U\Sigma V^T.$$

- The Moore–Penrose pseudoinverse is

$$A^+ = V\Sigma^+U^T,$$

where Σ^+ is obtained by:

- replacing every nonzero singular value σ_i by

$$\frac{1}{\sigma_i};$$

- transposing the rectangular shape of Σ ;
- leaving the zero singular values equal to zero.

For example, if

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \end{bmatrix},$$

then

$$\Sigma^+ = \begin{bmatrix} \frac{1}{\sigma_1} & 0 & 0 \\ \sigma_1 & \frac{1}{\sigma_2} & 0 \\ 0 & \frac{1}{\sigma_2} & 0 \end{bmatrix},$$

provided $\sigma_1, \sigma_2 > 0$.

If A is square and invertible, then

$$A^+ = A^{-1}.$$

If A is rectangular or rank deficient, A^+ still exists.

For the least-squares problem

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2,$$

the pseudoinverse gives

$$\boxed{\hat{\mathbf{x}} = A^+ \mathbf{b}}.$$

When there are several least-squares solutions, $A^+ \mathbf{b}$ gives the solution having the smallest Euclidean norm.

6.2.13 Low-rank approximation

Suppose

$$A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T.$$

A rank- k approximation of A is obtained by retaining only the first k terms:

$$\boxed{A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T}.$$

Equivalently,

$$\boxed{A_k = U_k \Sigma_k V_k^T},$$

where:

- U_k contains the first k left singular vectors;
- Σ_k contains the largest k singular values;
- V_k contains the first k right singular vectors.

The original matrix requires

$$mn$$

numbers to be stored.

The truncated SVD requires approximately

$$mk + k + nk = k(m + n + 1)$$

numbers.

When

$$k \ll \min(m, n),$$

this can greatly reduce storage requirements.

The truncated SVD is optimal in the sense that it gives the best rank- k approximation with respect to the Frobenius norm:

$$\|A - A_k\|_F \leq \|A - B\|_F$$

for every matrix B of rank at most k .

The Frobenius norm is defined by

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2}.$$

6.2.14 Choosing the number of components

The total squared information represented by the singular values can be measured using

$$\sum_{i=1}^r \sigma_i^2.$$

The proportion of information retained by the first k components is

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2}.$$

- A common strategy is to select the smallest k such that

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq 0.90$$

or

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq 0.95.$$

Thus, the reduced representation retains approximately 90% or 95% of the information.

6.2.15 Importance of SVD in machine learning

The SVD has many important applications in machine learning.

1. Dimensionality reduction A dataset may contain a large number of features, but many features may be correlated.

By retaining only the largest singular values and their singular vectors, the data can be represented in a lower-dimensional space:

$$A \approx A_k = U_k \Sigma_k V_k^T.$$

This reduces computational cost and can improve model performance.

2. Principal Component Analysis Let X be a centered data matrix. From the SVD,

$$X = U \Sigma V^T.$$

Then

$$X^T X = V \Sigma^T \Sigma V^T.$$

Therefore, the columns of V are the eigenvectors of $X^T X$.

These vectors are the principal directions used in PCA.

The eigenvalues of $X^T X$ are

$$\sigma_1^2, \sigma_2^2, \dots$$

Thus, PCA can be computed directly from the SVD without explicitly forming the covariance matrix.

This is often numerically more stable.

3. Image compression A grayscale image can be represented by a matrix

$$A \in \mathbb{R}^{m \times n},$$

- where each entry represents the intensity of a pixel. Instead of storing the entire image, one can store

$$U_k, \quad \Sigma_k, \quad V_k.$$

The approximation

$$A_k = U_k \Sigma_k V_k^T$$

can preserve the main visual features while using considerably less storage.

4. Noise reduction Noise frequently appears in directions associated with small singular values.

By removing the smallest singular values, one obtains

$$A_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T.$$

This can preserve the main signal while discarding part of the noise.

5. Recommendation systems Consider a matrix in which:

- rows represent users;
- columns represent products or movies;
- entries represent ratings.

A low-rank approximation can reveal hidden factors such as preferences for:

- action movies;
- comedy;
- music;
- sports;
- particular product categories.

These hidden factors can be used to predict missing ratings and generate recommendations.

6. Solving least-squares problems For

$$A\mathbf{x} \approx \mathbf{b},$$

the SVD gives the solution

$$\hat{\mathbf{x}} = A^+ \mathbf{b}.$$

This remains valid even when A is rectangular or rank deficient.

7. Detecting multicollinearity In regression, small singular values of the design matrix indicate nearly dependent features.

The condition number

$$\kappa(X) = \frac{\sigma_1}{\sigma_r}$$

helps measure the severity of this problem.

8. Matrix completion If some entries of a matrix are missing, the assumption that the underlying matrix has low rank can help estimate the missing values.

This idea is widely used in collaborative filtering and recommendation systems.

6.2.16 SVD in Python

In Python, the SVD can be computed using NumPy:

```
import numpy as np

A = np.array([
    [1.0, 2.0],
    [0.0, 1.0]
])

U, singular_values, VT = np.linalg.svd(A)

Sigma = np.zeros(A.shape)
np.fill_diagonal(Sigma, singular_values)

A_reconstructed = U @ Sigma @ VT

print("U:")
print(U)

print("Singular values:")
print(singular_values)

print("V transpose:")
print(VT)

print("Reconstructed matrix:")
print(A_reconstructed)
```

Notice that NumPy returns the singular values as a one-dimensional array rather than directly returning the matrix Σ .

For dimensionality reduction, a reduced SVD can be requested using

```
U, singular_values, VT = np.linalg.svd(
    A,
    full_matrices=False
)
```

For a rank- k approximation:

```
k = 1

Uk = U[:, :k]
Sk = np.diag(singular_values[:k])
VTk = VT[:k, :]

Ak = Uk @ Sk @ VTk
```

Exercise (QR, SVD, singular values, inverse).

(a) Let

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}.$$

Compute a *thin QR* factorization

$$X = QR,$$

with

$$Q \in \mathbb{R}^{3 \times 2}$$

having orthonormal columns and

$$R \in \mathbb{R}^{2 \times 2}$$

upper triangular.

Then solve the least-squares problem

$$\min_{\beta} \|X\beta - y\|_2$$

using

$$R\hat{\beta} = Q^T y.$$

(b) Let

$$B = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}.$$

Compute its singular values from $B^T B$ and give a short SVD description

$$B = U\Sigma V^T.$$

Numerical values may be used.

Decide whether B is invertible. If it is invertible, compute B^{-1} . Otherwise, determine its Moore–Penrose pseudoinverse B^+ .

(c) Consider the rank-deficient matrix

$$C = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}.$$

Use its singular values to determine:

- (a) the rank of C ;
- (b) whether C is invertible;
- (c) the role of C^+ when solving

$$C\mathbf{x} = \mathbf{b}.$$

Solution (summary).

1. (a) QR and LS via back-substitution):

Columns $a_1 = (1, 1, 0)^\top$, $a_2 = (1, 0, 1)^\top$.

$$r_{11} = \|a_1\| = \sqrt{2}, \quad q_1 = \frac{1}{\sqrt{2}}(1, 1, 0)^\top.$$

Orthogonalize a_2 against q_1 : $r_{12} = q_1^\top a_2 = \frac{1}{\sqrt{2}}$, $u_2 = a_2 - r_{12}q_1 = \left(\frac{1}{2}, -\frac{1}{2}, 1\right)^\top$, $r_{22} = \|u_2\| = \sqrt{\frac{3}{2}}$. Hence $q_2 = u_2/r_{22} \approx (0.4082, -0.4082, 0.8165)^\top$.

Therefore

$$Q = \begin{bmatrix} 0.7071 & 0.4082 \\ 0.7071 & -0.4082 \\ 0 & 0.8165 \end{bmatrix}, \quad R = \begin{bmatrix} 1.4142 & 0.7071 \\ 0 & 1.2247 \end{bmatrix}.$$

Solve $R\hat{\beta} = Q^\top y$. Since $Q^\top y = (0.7071, 1.2247)^\top$, back-substitution gives

$$\hat{\beta}_2 = \frac{1.2247}{1.2247} = 1, \quad \hat{\beta}_1 = \frac{0.7071 - 0.7071 \cdot 1}{1.4142} = 0.$$

$$\boxed{\hat{\beta} = (0, 1)^\top}$$

Interpretation: the best LS fit uses only the second column of X .

2. (b) SVD, singular values, and inverse of B):

$$B^T B = \begin{bmatrix} 1 & 2 \\ 2 & 5 \end{bmatrix}.$$

Its eigenvalues solve $\lambda^2 - 6\lambda + 1 = 0 \Rightarrow \lambda_{1,2} = 3 \pm 2\sqrt{2}$. Hence the singular values are

$$\sigma_1 = \sqrt{3 + 2\sqrt{2}} = \sqrt{2} + 1 \approx 2.4142, \quad \sigma_2 = \sqrt{3 - 2\sqrt{2}} = |\sqrt{2} - 1| \approx 0.4142.$$

Since both $\sigma_i > 0$, B is invertible. A short SVD is $B = U\Sigma V^\top$ with $\Sigma = \text{diag}(\sigma_1, \sigma_2)$, and V formed by the orthonormal eigenvectors of $B^T B$ (any numerically computed U, V with these σ_i is acceptable).

The (true) inverse is easy to check directly:

$$B^{-1} = \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix}.$$

(Because B is full rank, the pseudoinverse equals the inverse: $B^+ = B^{-1}$.)

3. (c) Rank-deficient C and the pseudoinverse):

For $C = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$, we have $\text{rank}(C) = 1$. Thus one singular value is positive and the other is $0 \Rightarrow C$ is *not* invertible. The SVD-based pseudoinverse $C^+ = V\Sigma^+U^\top$ (with Σ^+ inverting only the nonzero singular value) returns the *minimum-norm* solution of $Cx = b$ when the system is consistent, and the minimum-norm least-squares solution when it is not.

```
import numpy as np

# ----- Helpers -----
def back_substitute(R, b):
    """Solve R x = b for upper-triangular R (n x n)."""
    R = np.asarray(R)
    b = np.asarray(b)
    n = R.shape[0]
    x = np.zeros(n, dtype=float)
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - R[i, i+1:] @ x[i+1:]) / R[i, i]
    return x

# =====
# (a) QR least-squares for X beta ~ y
# =====
X = np.array([[1., 1.],
              [1., 0.],
              [0., 1.]]) # 3x2
y = np.array([1., 0., 1.])

# Thin (reduced) QR
Q, R = np.linalg.qr(X, mode="reduced") # Q: 3x2, R: 2x2, X = Q R
beta_via_qr = back_substitute(R, Q.T @ y)

# Cross-check with np.linalg.lstsq
beta_lstsq, residuals, rank, svals = np.linalg.lstsq(X, y, rcond=None)

print("----- (a) QR Least Squares -----")
print("Q =\n", Q)
print("R =\n", R)
print("Q^T y =", Q.T @ y)
print("beta (via QR) =", beta_via_qr)
print("beta (np lstsq) =", beta_lstsq)
print("residual norm =", np.linalg.norm(X @ beta_via_qr - y))
print()

# =====
# (b) SVD, singular values, and inverse for B
# =====
B = np.array([[1., 2.],
              [0., 1.]])
```

```

# Singular values from  $B^T B$ 
BtB = B.T @ B
eigs = np.linalg.eigvalsh(BtB) # sorted eigenvalues
sing_from_BtB = np.sqrt(eigs) # singular values

# Full SVD ( $U$ ,  $\Sigma$ ,  $Vt$ )
U, s, Vt = np.linalg.svd(B, full_matrices=True)

# Inverse (exists because  $\text{rank}(B)=2$ )
B_inv = np.linalg.inv(B)
I_check = B @ B_inv

print("----- (b) SVD & Inverse of B -----")
print("BT B =\n", BtB)
print("Eigenvalues(BT B) =", eigs)
print("Singular values (sqrt eigs) =", sing_from_BtB)
print("SVD singular values =", s)
print("V (right-singular vecs) =\n", Vt.T)
print("B-1 =\n", B_inv)
print("B @ B-1 ~ I ? ->", np.allclose(I_check, np.eye(2)))
# Pseudoinverse equals inverse for full-rank square B
print("pinv(B) == inv(B) ? ->", np.allclose(np.linalg.pinv(B), B_inv))
print()

# =====
# (c) Rank-deficient C: pseudoinverse and min-norm solutions
# =====
C = np.array([[1., 2.],
              [2., 4.]]) # rank 1

rank_C = np.linalg.matrix_rank(C)
U_C, s_C, Vt_C = np.linalg.svd(C, full_matrices=True)
C_pinv = np.linalg.pinv(C)

# Choose a consistent b (in column space of C):  $[3, 6] = 3 * [1, 2]$ 
b_consistent = np.array([3., 6.])
x_min_norm_consistent = C_pinv @ b_consistent # exact solution with minimum
norm

# Choose an inconsistent b' (not in column space):  $[1, 0]$ 
b_inconsistent = np.array([1., 0.])
x_min_norm_ls = C_pinv @ b_inconsistent # min-norm least-squares solution

print("----- (c) Rank-deficient C -----")
print("rank(C) =", rank_C)
print("Singular values of C =", s_C)
print("C+ (pseudoinverse) =\n", C_pinv)

print("\nConsistent system C x = b with b=[3,6]T:")
print("x_min_norm =", x_min_norm_consistent)
print("Check Cx =", C @ x_min_norm_consistent, " (should equal b)")

```

```
print("\nInconsistent system C x = b' with b'=[1,0]^T (least squares):")
print("x_min_norm_ls =", x_min_norm_ls)
print("Residual norm =", np.linalg.norm(C @ x_min_norm_ls - b_inconsistent))
```

Takeaways.

- **QR** solves LS via $R\hat{\beta} = Q^T y$ (no explicit inverse).
- **SVD** exposes rank/conditioning; singular values $\downarrow \Rightarrow$ ill-conditioning.
- **Pseudoinverse** (X^+) gives the minimum-norm solution when X is rank-deficient.

7 Application: Principal Component Analysis (PCA)

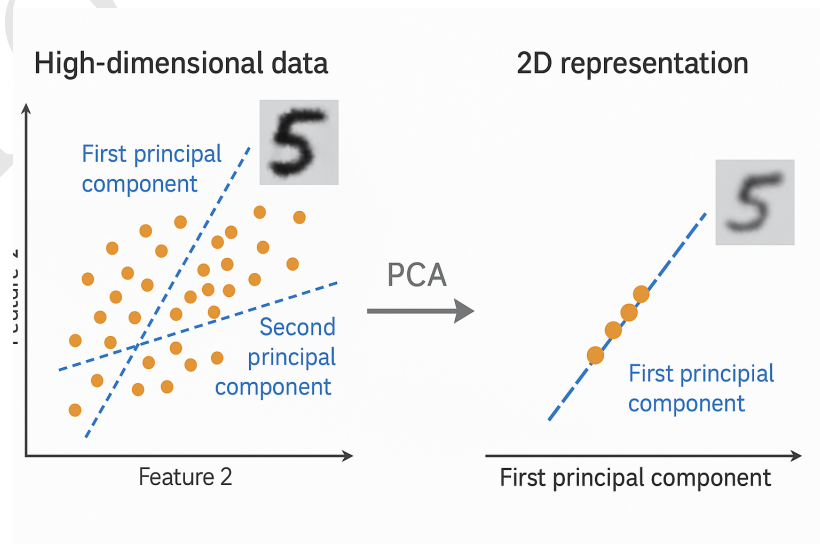
Modern datasets are often high-dimensional: an image with 100×100 pixels has 10,000 features, and a text document represented by word counts may have tens of thousands of dimensions. High-dimensional data are difficult to visualize, computationally expensive to process, and often contain redundancy because many features are correlated.

Principal Component Analysis (PCA) is a linear transformation technique that reduces dimensionality by finding new orthogonal axes (principal components) along which the data vary the most. Instead of keeping all original features, PCA projects the data onto the leading components, preserving as much information (variance) as possible.

7.1 Geometric and Statistical Intuition

The first principal component is the direction in feature space along which the variance of the data is maximized. The second principal component is orthogonal to the first and captures the largest remaining variance, and so on.

Equivalently, PCA finds the linear transformation $Y = WX$ (with W built from eigenvectors of the covariance matrix) that maximizes variance in the projected data while minimizing information loss.



7.2 Algorithmic Steps

Given a centered data matrix X (n samples, d features):

1. Compute the covariance matrix

$$C = \frac{1}{n-1} X^T X.$$

2. Solve the eigenvalue problem $C\mathbf{v} = \lambda\mathbf{v}$ to obtain eigenvalues λ_i and eigenvectors $\mathbf{v}_i$.
3. Sort eigenvectors in decreasing order of eigenvalues (largest variance first).
4. Choose the first k eigenvectors and project data:

$$Z = XW_k, \quad W_k = [\mathbf{v}_1, \dots, \mathbf{v}_k].$$

7.3 Worked Numerical Example (2D Data)

Suppose we have three 2D points:

$$(2, 0), \quad (0, 2), \quad (3, 3).$$

Step 1. Center the data by subtracting the mean $(1.67, 1.67)$. The centered points are:

$$(0.33, -1.67), \quad (-1.67, 0.33), \quad (1.33, 1.33).$$

Step 2. Compute the covariance matrix:

$$C = \frac{1}{3-1} \begin{bmatrix} 0.33 & -1.67 & 1.33 \\ -1.67 & 0.33 & 1.33 \end{bmatrix} \begin{bmatrix} 0.33 & -1.67 & 1.33 \\ -1.67 & 0.33 & 1.33 \end{bmatrix}^T = \begin{bmatrix} 2.33 & 1.17 \\ 1.17 & 2.33 \end{bmatrix}.$$

Step 3. Solve $\det(C - \lambda I) = 0$:

$$\det \begin{bmatrix} 2.33 - \lambda & 1.17 \\ 1.17 & 2.33 - \lambda \end{bmatrix} = 0 \quad \Rightarrow \quad (\lambda - 3.5)(\lambda - 1.17) = 0.$$

Eigenvalues: $\lambda_1 = 3.5$, $\lambda_2 = 1.17$.

Step 4. Eigenvectors: For $\lambda_1 = 3.5$, eigenvector $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ (direction along the diagonal). For $\lambda_2 = 1.17$, eigenvector $\mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

Step 5. Interpretation: The data vary most along the direction $(1, 1)$, which explains about $\frac{3.5}{3.5+1.17} \approx 75\%$ of the variance. Projecting onto $\mathbf{v}_1$ reduces 2D data to 1D while keeping most information.

```
import numpy as np
import matplotlib.pyplot as plt

# -----
# Data: three 2D points
# -----
X = np.array([[2.0, 0.0],
              [0.0, 2.0],
```

```

        [3.0, 3.0])) # shape (3, 2)
n = X.shape[0]

# 1) Center the data
mu = X.mean(axis=0)
Xc = X - mu

print("Mean (should be close to [1.6667, 1.6667]):", mu)
print("Centered data:\n", Xc)

# 2) Sample covariance (divide by n-1)
# np.cov with rowvar=False returns the (biased/unbiased) covariance across
# columns
C = np.cov(X, rowvar=False, bias=False) # same as (Xc.T @ Xc)/(n-1)
print("Sample covariance matrix C =\n", C)

# 3a) PCA via eigen-decomposition of covariance
evals, evecs = np.linalg.eigh(C) # returns ascending eigenvalues
idx = np.argsort(evals)[::-1] # sort descending
evals = evals[idx]
evecs = evecs[:, idx]
print("Eigenvalues (descending) =", evals)
print("Eigenvectors (columns) =\n", evecs)

# 3b) PCA via SVD of centered data
# Xc = U S Vt, PCs are columns of V, singular values s relate to variance by s
# ^2/(n-1)
U, s, Vt = np.linalg.svd(Xc, full_matrices=False)
V = Vt.T
explained_var = (s**2) / (n - 1)
print("Singular values =", s)
print("Explained variances via SVD (s^2/(n-1)) =", explained_var)
print("Principal directions via SVD (columns of V) =\n", V)

# 4) Project onto first principal component (PC1)
pc1 = evecs[:, 0] # same direction as V[:, 0] up to sign
X_proj_1d = Xc @ pc1 # coordinates along PC1
X_proj_2d = np.outer(X_proj_1d, pc1) # back to 2D in centered coords

print("1D coordinates on PC1:", X_proj_1d)

# -----
# 5) Plots
# (a) Centered points with principal axes
# (b) Projections onto PC1 (line)
# -----

# Helper: draw a principal axis as a segment centered at origin
def draw_axis(ax, direction, length=2.0, **kwargs):
    d = direction / np.linalg.norm(direction)
    p1 = -length * d

```

```

    p2 = length * d
    ax.plot([p1[0], p2[0]], [p1[1], p2[1]], **kwargs)

# (a) Centered data with PCs
fig, ax = plt.subplots(figsize=(5, 5))
ax.scatter(Xc[:, 0], Xc[:, 1], s=60, label="Centered points")

# Principal axes (scale by sqrt of eigenvalues to suggest variance)
draw_axis(ax, evects[:, 0] * np.sqrt(evals[0]), length=2.0, color="C1",
          linewidth=2, label="PC1")
draw_axis(ax, evects[:, 1] * np.sqrt(evals[1]), length=2.0, color="C2",
          linewidth=2, label="PC2")

# Coordinate axes for reference
ax.axhline(0, color="gray", linewidth=0.8)
ax.axvline(0, color="gray", linewidth=0.8)

ax.set_aspect("equal", adjustable="box")
ax.set_title("PCA on centered data: points and principal axes")
ax.set_xlabel("x (centered)")
ax.set_ylabel("y (centered)")
ax.legend(loc="best")
plt.show()

# (b) Projections on PC1 (show original centered points and their drops to PC1
#     line)
fig, ax = plt.subplots(figsize=(5, 5))
ax.scatter(Xc[:, 0], Xc[:, 1], s=60, label="Centered points")
ax.scatter(X_proj_2d[:, 0], X_proj_2d[:, 1], s=60, marker="x", label="
    Projections on PC1")

# Draw PC1 axis line
draw_axis(ax, pc1, length=3.0, color="C1", linewidth=2, label="PC1 line")

# Connect each point to its projection
for i in range(n):
    ax.plot([Xc[i, 0], X_proj_2d[i, 0]],
            [Xc[i, 1], X_proj_2d[i, 1]],
            linestyle="--", color="gray", linewidth=1)

ax.axhline(0, color="gray", linewidth=0.8)
ax.axvline(0, color="gray", linewidth=0.8)
ax.set_aspect("equal", adjustable="box")
ax.set_title("Projection onto PC1 (1D)")
ax.set_xlabel("x (centered)")
ax.set_ylabel("y (centered)")
ax.legend(loc="best")
plt.show()

# -----
# 6) Variance explained

```

```
# -----
total_var = evals.sum()
explained_pc1 = evals[0] / total_var
explained_pc2 = evals[1] / total_var
print("Explained variance ratio: PC1 =", explained_pc1, ", PC2 =",
      explained_pc2)
```

7.4 Interpretation in Machine Learning

PCA is widely used:

- **Visualization:** Reducing thousands of features to 2 or 3 components makes it possible to plot and explore patterns.
- **Noise reduction:** By discarding components with small eigenvalues, we remove directions dominated by noise.
- **Preprocessing:** PCA is often applied before clustering or regression to avoid multicollinearity.
- **Face recognition:** Images are represented as high-dimensional vectors. PCA extracts “eigenfaces,” which capture the main modes of variation, enabling fast and efficient recognition.
- **Recommender systems:** User–item matrices can be compressed using PCA or SVD, revealing latent factors that drive preferences.

8 Exercice - Blood Pressure Data: PCA Interpretation

We consider the following dataset of blood pressure measurements (systolic and diastolic):

$$X = \begin{bmatrix} 126 & 78 \\ 128 & 80 \\ 128 & 82 \\ 130 & 82 \\ 130 & 84 \\ 132 & 86 \end{bmatrix}$$

- Center the data.** Compute the column means of systolic and diastolic blood pressure, then subtract them to obtain the centered data matrix.
- Covariance matrix.** Compute the covariance matrix

$$C = \frac{1}{n-1} X_c^\top X_c$$

where X_c is the centered data.

- Eigen-decomposition.** Find the eigenvalues and eigenvectors of C .

- (d) **Interpretation.** - Which eigenvalue is largest? What percentage of the total variance does it explain? - What does the associated eigenvector represent in terms of blood pressure? - Relate your findings to the correlation between systolic and diastolic pressure.

9 Conclusion

This chapter has established the linear algebraic foundations of machine learning. Vectors and matrices provide the language for representing data and transformations. Matrix arithmetic describes how features combine. Eigenvalues and eigenvectors reveal invariant directions, while matrix decompositions such as the SVD give powerful tools for analyzing structure and compressing data. PCA illustrates the culmination of these ideas in a widely used method for dimensionality reduction.

Together, these concepts form a crucial bridge between mathematics and practical machine learning algorithms, preparing students to implement and understand them in Python.